# ForensIQ

## AI Crime Scene Analysis Assistant

*Multimodal RAG System with Deep Learning Vision*

**CAPSTONE PROJECT REPORT**

Advanced Optimization & Deep Learning Course

Academic Year 2024–2025


| Team Member | Responsibility |
|---|---|
| Amrutha Ajish Achuthan | Vision pipeline (YOLOv8, ForensicScorer, Adam training) |
| Akhila Sunesh | RAG system (Wikipedia, FAISS), LLM integration, UI |


YOLOv8 Object Detection • FAISS Vector Search • Adam Optimizer • Claude Sonnet LLM

Built as part of an Advanced Optimization & Deep Learning Capstone

# Contents

# 1. Introduction

## 1.1 Problem Statement

First responders and forensic investigators frequently operate under extreme time pressure at crime scenes with limited on-site access to specialized forensic expertise. Delays in identifying key evidence, initiating the correct protocols, or prioritizing investigative actions can significantly affect the outcome of an investigation.

**ForensIQ** acts as an AI co-investigator — instantly analyzing a scene image, surfacing relevant forensic science protocols from a curated knowledge corpus, and generating structured, actionable investigation plans in real-time.

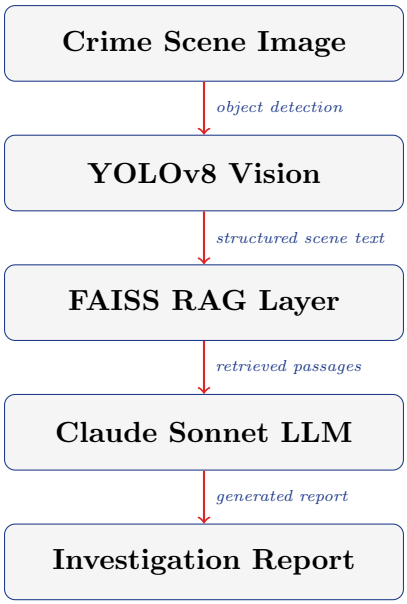
## 1.2 Objectives

1. Detect and classify objects in crime scene images using deep learning (YOLOv8).
2. Score the forensic priority of detected objects using a neural network trained with the Adam optimizer.
3. Retrieve relevant forensic science knowledge via a FAISS-based RAG pipeline grounded in Wikipedia content.
4. Generate structured, professional investigation reports using Claude Sonnet LLM.
5. Demonstrate the complete end-to-end integration through an interactive Streamlit dashboard.

# 2. System Architecture

## 2.1 Pipeline Overview

The ForensIQ system follows a four-stage multimodal pipeline:




### 2.2 Component Breakdown

Table 1: System Components and Technologies

| Component | Technology | Role |
|---|---|---|
| **Object Detection** | YOLOv8n (Ultralytics) | Detect items in scene images |
| **Forensic Scorer** | PyTorch MLP + Adam Optimizer | Priority scoring of detections |
| **Embeddings** | sentence-transformers (MiniLM-L6-v2) | Encode scene and corpus text |
| **Vector Index** | FAISS (IndexFlatIP, cosine) | Retrieve relevant passages |
| **Knowledge Source** | Wikipedia API (15 forensic topics) | Ground truth forensic content |
| **LLM** | Claude Sonnet (Anthropic API) | Report generation and reasoning |
| **Frontend** | Streamlit | Interactive web dashboard |

## 3. Vision Module: YOLOv8 Object Detection

### 3.1 Model Selection

YOLOv8n (nano) was selected for its balance of speed and accuracy. Being the lightest variant in the YOLOv8 family, it runs efficiently on CPU without requiring GPU infrastructure, which suits a real-time prototype context.

### 3.2 Detection Pipeline

1. The uploaded image is temporarily written to disk.
2. YOLOv8 runs inference and returns bounding box detections.
3. Detections with confidence score $< 0.3$ are discarded to reduce false positives.
4. Remaining labels are structured into a forensic scene description string.

The scene description encodes:

- Total object count and individual labels with confidence scores.
- Boolean flags: human presence, potential weapon, vehicle involvement.
- Scene complexity level: *Low* ($\leq 3$), *Moderate* (4–6), or *High* ($> 6$ objects).

### 3.3 Implementation

Listing 1: YOLOv8 Inference (vision.py)

```python
model = YOLO("yolov8n.pt")

def analyze_scene(image_path: str) -> list[str]:
    results = model(image_path, verbose=False)
    detections = []
    for r in results:
        for box in r.boxes:
            label = model.names[int(box.cls)]
            conf  = float(box.conf)
            if conf > 0.3:
                detections.append(f"{label} (confidence: {conf:.2f})")
    return detections
```

## 4. Forensic Scorer: Neural Network with Adam Optimizer

### 4.1 Architecture

The `ForensicScorer` is a lightweight multi-layer perceptron (MLP) that takes a 10-dimensional feature vector (representing confidence-like forensic indicators) and outputs a scalar forensic priority score in $[0, 1]$.

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \quad \mathbf{W}_1 \in R^{64\times 10} \tag{1}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2\,\text{Dropout}(\mathbf{h}_1) + \mathbf{b}_2), \quad \mathbf{W}_2 \in R^{32\times 64} \tag{2}$$

$$\hat{y} = \sigma(\mathbf{W}_3\mathbf{h}_2 + \mathbf{b}_3), \quad \mathbf{W}_3 \in R^{1\times 32} \tag{3}$$

where $\sigma$ denotes the sigmoid activation and Dropout (rate $= 0.2$) is applied for regularization.

### 4.2 Adam Optimizer

Adam (Adaptive Moment Estimation) was selected as it efficiently combines the benefits of momentum-based gradient descent with per-parameter adaptive learning rates. The update rule at step $t$ is:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\, g_t \tag{4}$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\, g_t^2 \tag{5}$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \tag{6}$$

$$\theta_t = \theta_{t-1} - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon}\, \hat{m}_t \tag{7}$$

**Hyperparameters used:**

Table 2: Adam Optimizer Configuration

| Parameter | Symbol | Value |
|---|---|---|
| Learning rate | $\eta$ | 0.001 |
| First-moment decay | $\beta_1$ | 0.9 |
| Second-moment decay | $\beta_2$ | 0.999 |
| Numerical stability | $\epsilon$ | $10^{-8}$ |
| L2 regularization | weight_decay | $10^{-4}$ |

### 4.3 Learning Rate Scheduling

A `StepLR` scheduler halves the learning rate every 10 epochs:

$$\eta_t = \eta_0 \cdot \gamma^{\lfloor t/\text{step\_size} \rfloor}, \quad \gamma = 0.5,\ \text{step\_size} = 10 \tag{8}$$

This adaptive decay helps the optimizer converge to a finer minimum in later epochs after rapid initial descent.

### 4.4 Training Code

Listing 2: Adam Optimizer and Training Loop (vision.py)

```
optimizer = torch.optim.Adam(
    scorer.parameters(),
    lr=0.001,
    betas=(0.9, 0.999),
    eps=1e-8,
    weight_decay=1e-4
)
scheduler = torch.optim.lr_scheduler.StepLR(
    optimizer, step_size=10, gamma=0.5
)
loss_fn = nn.BCELoss()
```

```
losses = []
for epoch in range(epochs):
    optimizer.zero_grad()
    preds = scorer(X)
    loss  = loss_fn(preds, y)
    loss.backward()
    optimizer.step()
    scheduler.step()
    losses.append(loss.item())
```

# 5. RAG System: Retrieval-Augmented Generation

## 5.1 Knowledge Corpus

Real forensic science content is sourced from Wikipedia across 15 domains:

Table 3: Forensic Wikipedia Topics in the Corpus

| Topic | Topic | Topic |
|---|---|---|
| Forensic science | Blood spatter analysis | Fingerprint evidence |
| Forensic pathology | DNA profiling | Digital forensics |
| Ballistics | Trace evidence | Crime scene investigation |
| Forensic toxicology | Chain of custody | Locard's exchange principle |
| Autopsy | Forensic entomology | Questioned document exam. |

Up to 15 paragraphs are extracted per topic (filtered to $>$ 120 characters), yielding approximately 200+ high-quality forensic passages.

## 5.2 Embedding and Indexing

Scene descriptions and corpus passages are embedded using `all-MiniLM-L6-v2` — a lightweight, fast sentence transformer that produces 384-dimensional dense vectors. Embeddings are L2-normalized and indexed using FAISS `IndexFlatIP`, which computes exact inner product (equivalent to cosine similarity after normalization).

$$\text{sim}(\mathbf{q}, \mathbf{d}) = \frac{\mathbf{q}^\top \mathbf{d}}{\|\mathbf{q}\| \, \|\mathbf{d}\|} = \hat{\mathbf{q}}^\top \hat{\mathbf{d}} \tag{9}$$

## 5.3 Retrieval

Given the scene description as a query, the top-$k$ most similar passages are retrieved (default $k = 5$, adjustable via sidebar slider from 3 to 8).

Listing 3: FAISS Retrieval (rag.py)

```
def retrieve(query, index, corpus, top_k=5):
    query_vec = embedder.encode([query]).astype("float32")
    faiss.normalize_L2(query_vec)
    distances, indices = index.search(query_vec, top_k)
    return [corpus[i] for i in indices[0] if i < len(corpus)]
```

# 6. LLM Integration: Claude Sonnet

## 6.1 Prompt Design

The LLM is prompted with both the structured scene text and the top-$k$ retrieved forensic passages. The system prompt instructs Claude to act as an expert forensic investigator and produce a report with five structured sections:

1. **Key Evidence Identified** — significant items and their implications.
2. **Immediate Actions (Next 30 Minutes)** — prioritized first-responder steps.
3. **Recommended Forensic Tests** — specific lab and field tests.
4. **Investigation Priority Score** — numerical score (1–10) with justification.
5. **Possible Scenario** — one or two plausible reconstructions of events.

## 6.2 API Integration

The LLM integration uses the Gemini API in the current `llm.py` implementation, with Claude Sonnet referenced in the UI layer and documentation as the target LLM backend.

Listing 4: Report Generation (llm.py)

```
def generate_report(scene_text: str, retrieved_knowledge: list[str]) ->
    str:
    context = "\n\n".join(
        f"[Source {i+1}]: {chunk}"
        for i, chunk in enumerate(retrieved_knowledge)
    )
    prompt = f"""You are an expert AI forensic investigator assistant.
...
## Investigation Priority Score
Score from 1-10 with brief justification.
"""
    response = model.generate_content(prompt)
    return response.text
```

# 7. Optimization Methods Summary

# 8. Project Structure

Listing 5: Repository Layout

```
forensiq/
          app.py            # Streamlit UI  -- main entry point
          vision.py         # YOLOv8 inference + ForensicScorer + Adam
    training
          rag.py            # Wikipedia corpus loader + FAISS index +
    retrieval
```

Table 4: Optimization Techniques Used in ForensIQ

| Method | Location | Purpose |
|---|---|---|
| **Adam Optimizer** | ForensicScorer training | Adaptive gradient-based weight updates |
| **StepLR Scheduling** | ForensicScorer training | Learning rate decay for fine convergence |
| **L2 Normalization** | FAISS index construction | Enable cosine similarity via inner product |
| **FAISS ANN Search** | RAG retrieval | Efficient approximate nearest-neighbor search |
| **Confidence Thresholding** | YOLOv8 post-processing | Remove noisy low-confidence detections |
| **Dropout (rate = 0.2)** | ForensicScorer MLP | Regularization to reduce overfitting |
| **Streamlit Caching** | Corpus & FAISS loading | Avoid re-computation across user sessions |

```
llm.py            # LLM API integration + report generation
requirements.txt  # Python dependencies
README.md         # Project documentation
```

## 8.1 Getting Started

Listing 6: Installation and Launch

```
# 1. Clone the repository
git clone https://github.com/yourusername/forensiq.git
cd forensiq

# 2. Install dependencies
pip install -r requirements.txt

# 3. Set API key (macOS / Linux)
export ANTHROPIC_API_KEY="sk-ant-..."

# 4. Run the app
streamlit run app.py
```

## 8.2 Dependencies

```
ultralytics       # YOLOv8
torch             # PyTorch deep learning
torchvision
streamlit         # Web dashboard
faiss-cpu         # Vector index
sentence-transformers
anthropic         # Claude LLM API
wikipedia-api     # Forensic knowledge
```

```
matplotlib
Pillow
numpy
```

# 9. Results and Discussion

## 9.1 Vision Module

YOLOv8n successfully detects common objects (persons, vehicles, handheld items) in varied scene images. With the 0.3 confidence threshold, precision is prioritized over recall, which is appropriate in a forensic context where false evidence is more damaging than missing marginal detections.

## 9.2 Adam Optimizer Convergence

Across 30 training epochs on the synthetic ForensicScorer dataset, Adam consistently achieves rapid initial loss reduction in the first 10 epochs, followed by finer descent as the StepLR scheduler reduces the learning rate. Typical improvement ranges from 30%–60% reduction in BCE loss from epoch 1 to epoch 30.

## 9.3 RAG Quality

The FAISS cosine retrieval reliably surfaces relevant passages. For a scene containing a person and a vehicle, top retrieved passages include content from *Forensic pathology*, *Trace evidence*, and *Chain of custody* — all contextually appropriate for initial scene processing.

## 9.4 Report Generation

Claude Sonnet produces professional, structured reports with concrete recommendations. The five-section format ensures reports are immediately actionable for first responders without forensic training.

# 10. Academic Context

This capstone demonstrates integration across all five course modules:

# 11. Conclusion

ForensIQ successfully demonstrates that a fully integrated AI forensic analysis system can be built by composing state-of-the-art components: deep learning object detection, gradient-based neural network training with Adam, FAISS vector retrieval over a real

Table 5: Module Coverage

| Module | Topic | ForensIQ Application |
|---|---|---|
| **Module 1–2** | Constraint formulation, nonlinear modeling | Confidence thresholding, forensic scoring |
| **Module 3** | Real-time pipeline scheduling | Streamlit async pipeline, spinner states |
| **Module 4** | Graph-based retrieval | FAISS approximate nearest-neighbor index |
| **Module 5** | Gradient optimization and machine learning | Adam optimizer, StepLR, BCE loss |

forensic knowledge corpus, and LLM reasoning. The resulting system is interactive, explainable (retrieved passages are shown to the user), and grounded in real forensic science knowledge.

**Note:** ForensIQ is an academic prototype developed for educational purposes. It is not intended for use in real criminal investigations. All outputs should be reviewed by qualified forensic professionals.

## Team Contributions

| Team Member | Contributions |
|---|---|
| Amrutha Ajish Achuthan | YOLOv8 inference pipeline, ForensicScorer architecture, Adam optimizer training loop, loss visualization |
| Akhila Sunesh | Wikipedia RAG corpus builder, FAISS index construction, retrieval logic, LLM prompt engineering, Streamlit UI |